Sistemas Operativos
III Cuatrimestre 2026

LABORATORIO 2

Análisis de bloques de código dinámicos y vigilancia de archivos en vivo.

Todo funciona/se puede hacer en Linux, MacOS o Windows 10/11 (**con wsl o gitbash**)

Objetivos de aprendizaje. Al terminar, la persona estudiante podrá:

1. Leer porciones de un archivo grande por líneas o por rango (head, tail, sed -n).
2. Filtrar líneas con grep usando opciones y expresiones regulares, y redirigir la salida a archivos (>, >>, 2>, tee).
3. Seguir un log que crece en tiempo real (tail -f / tail -F) y capturar solo lo relevante.
4. Filtrar eventos por rango de tiempo con awk.
5. Escribir scripts Bash con parámetros, validaciones, códigos de salida y reportes en archivo.

Materiales (entregados junto a esta guía):

- simulador_log.py — genera un app.log (o web.log tipo Apache) en lote o en vivo. Requiere Python 3.8+, sin librerías extra.

QUE DEBE ENTREGAR:

Evidencia: como mínimo:

- screenshots de cada punto checkpoint
- los scripts que modifique o escriba.
- Todo bien organizado y CLARO, que yo no tenga que adivinar.

INTODUCCIÓN

¿QUÉ ES UN LOG?

Un log es un archivo que registra eventos de un programa o servicio en ejecución, tienen un formato parecido a:

```
2026-10-15 11:22:00 [INFO] pagos[4527]: Pedido #2519 creado correctamente
2026-10-15 11:22:07 [ERROR] auth[3117]: Login fallido para usuario hugo desde 45.33.12.9
```

Luego veremos que con la herramienta aws, esto se puede analizar como campos: $1 fecha, $2 hora, $3 nivel, $4 servicio[PID].

PREPARACIÓN

Sistema	**Terminal recomendada**	**Nota**
Linux	Terminal (bash)	Nada extra

macOS	Terminal (zsh)	Los scripts usan bash script.sh
Windows 10/11 (opción A)	WSL con Ubuntu: wsl --install en PowerShell como administrador y reiniciar	**Es Linux real: la mejor opción**
Windows 10/11 (opción B)	Git Bash (incluido en Git for Windows) (si no puede instalar WSL)	tail -F y grep funcionan igual

1. Crear la carpeta de trabajo y copiar ahí `simulador_log.py`:

```
mkdir -p ~/lab_logs && cd ~/lab_logs


ls
```

2. Generar un historial de 500 líneas reproducible (todos tendrán el mismo archivo gracias a `--semilla`):

```
python3 simulador_log.py --lineas 500 --semilla 42 --rafagas
```

3. Ver la ayuda del simulador:

```
python3 simulador_log.py --help
```

Opciones del simulador

Opción	Qué hace	Ejemplo
--lineas N	Modo lote: escribe N líneas y termina	--lineas 1000
--tasa X	Modo continuo: X líneas por segundo (por defecto 2)	--tasa 10
--duracion S	Detiene el modo continuo tras S segundos	--duracion 60
--rafagas	Alterna periodos tranquilos con ráfagas de errores	--rafagas
--formato web	Formato access.log de Apache/Nginx	--formato web --archivo web.log
--rotar N	Rota el archivo cada N líneas (app.log → app.log.1)	--rotar 50
--anexar	Agrega al archivo en vez de vaciarlo	--anexar
--semilla N	Resultados reproducibles	--semilla 42

4. Checkpoint: ls -l muestra app.log y wc -l app.log devuelve 500 y ayuda del generador.

Actividad 1 — Lectura por bloques (25 min)

Un log real puede tener millones de líneas; nunca se abre completo. Se lee por trozos: el inicio, el final o un rango.

Demostración guiada (pruebe estos comandos en su terminal, capture algunos ← checkpoint)

```
wc -l app.log            # ¿cuántas líneas hay?
head -n 5 app.log        # las 5 primeras
tail -n 5 app.log        # las 5 últimas (lo más reciente)
tail -n +495 app.log     # desde la línea 495 hasta el final
sed -n '100,110p' app.log   # solo las líneas 100 a 110
head -n 110 app.log | tail -n 11   # lo mismo, combinando comandos
less app.log             # paginar: espacio avanza, /ERROR busca, q sale
```

Ejercicios

1. Muestre las líneas 250 a 260 de dos formas distintas.
2. Guarde las últimas 50 líneas en `ultimas50.txt` y confirme con `wc -l` que tiene 50.
3. Divida el log en bloques de 100 líneas: `split -l 100 app.log bloque_` y luego `ls bloque_*`. ¿Cuántos archivos se crearon y por qué?
4. Recorra los bloques con un ciclo y muestre la primera línea de cada uno:

```
for b in bloque_*; do echo "== $b"; head -n 1 "$b"; done
```

☐ Checkpoint: `split` produjo 5 archivos y `ultimas50.txt` tiene 50 líneas. Y el screenshot de la demostración guiada.

Actividad 2 — grep y salida a archivos (35 min)

`grep` imprime solo las líneas que coinciden con un patrón; la redirección decide si ese resultado va a la pantalla o a un archivo.

Opciones de grep que se usan en el laboratorio

Opción	Efecto	Ejemplo
`-i`	Ignora mayúsculas/minúsculas	`grep -i error app.log`
`-c`	Cuenta coincidencias en vez de mostrarlas	`grep -c '\[ERROR\]' app.log`
`-v`	Invierte: líneas que NO coinciden	`grep -v '\[DEBUG\]' app.log`
`-n`	Muestra el número de línea	`grep -n CRITICAL app.log`
`-E`	Expresiones regulares extendidas (alternativa con barra vertical, `+`, `{}`)	`grep -E '\[(ERROR|CRITICAL)\]' app.log`
`-o`	Imprime solo la parte que coincide	`grep -oE '[0-9]+(\.[0-9]+){3}' app.log`
`-A n` / `-B n` / `-C n`	n líneas de contexto después / antes / ambos	`grep -B 2 CRITICAL app.log`
`-m n`	Se detiene tras n coincidencias	`grep -m 3 ERROR app.log`

Redirección

Símbolo	Qué hace
`> archivo`	Crea o sobrescribe el archivo con la salida
`>> archivo`	Agrega al final del archivo
`2> archivo`	Guarda los mensajes de error (stderr)
`| tee archivo`	Muestra en pantalla y guarda a la vez

Demostración guiada

```
mkdir -p salidas
grep '\[ERROR\]' app.log > salidas/errores.txt
grep '\[CRITICAL\]' app.log >> salidas/errores.txt   # se agrega, no se borra
wc -l salidas/errores.txt
grep 'Login fallido' app.log | tee salidas/logins.txt | wc -l
grep ERROR archivo_que_no_existe.log 2> salidas/fallos.txt
cat salidas/fallos.txt
```

Ejercicios

2. Compare `grep -c ERROR app.log` con `grep -c '\[ERROR\]' app.log`. ¿Salen iguales? Explique por qué el segundo es más preciso.

3. Guarde en `salidas/sin_debug.txt` todo el log excepto las líneas DEBUG.
4. Cuente cuántas líneas hay de cada nivel con un ciclo:

```
for n in DEBUG INFO WARN ERROR CRITICAL; do
  echo "$n: $(grep -c "\[$n\]" app.log)"
done > salidas/conteo_niveles.txt
```

4. **Investigación de seguridad.** ¿Desde qué IPs hay más logins fallidos? Guarde el ranking en `salidas/ips_sospechosas.txt`:

```
grep 'Login fallido' app.log | grep -oE '[0-9]+(\.[0-9]+){3}' | sort | uniq -c | sort -rn > salidas/ips_sospechosas.txt
```

5. ¿Qué servicio produce más errores? Pista: `awk '{print $4}'` extrae el campo y `cut -d'[' -f1` quita el PID.
6. Muestre cada evento CRITICAL con las 2 líneas que lo precedieron. ¿Se ve alguna señal previa (WARN, reintentos)?

☐ Checkpoint: la carpeta `salidas/` tiene al menos 5 archivos y en el ranking aparecen arriba IPs externas (no 192.168.x.x). adjunte además los archivos de salida en un link a un directorio de onedrive.

Actividad 3 — Seguimiento en tiempo real (30 min)

`tail -f` mantiene el archivo abierto y muestra cada línea nueva en cuanto se escribe; es la herramienta básica de monitoreo en servidores. Para este bloque se usan **dos terminales** lado a lado.

Demostración guiada

Terminal 1 — el "servidor" escribe en vivo:

```
python3 simulador_log.py --tasa 3 --rafagas
```

Terminal 2 — la persona operadora observa:

```
tail -f app.log                          # todo lo nuevo (Ctrl+C para salir)
tail -n 0 -f app.log                     # solo lo nuevo, sin las 10 líneas in
iciales
tail -f app.log | grep --line-buffered -E '\[(ERROR|CRITICAL)\]'
tail -f app.log | grep --line-buffered CRITICAL | tee salidas/criticos_vivo.t
xt
```

`--line-buffered` obliga a `grep` a entregar cada línea de inmediato; sin él, cuando la salida va a un archivo o a otro comando, `grep` acumula en memoria y el archivo parece vacío por un rato.

`tail -f` contra `tail -F` (rotación de logs)

Los servidores renombran el log cuando crece (`app.log` → `app.log.1`) y empiezan uno nuevo. `tail -f` se queda pegado al archivo viejo; `tail -F` vuelve a abrir el archivo por su nombre.

5. Terminal 1: `python3 simulador_log.py --tasa 5 --rotar 40`
6. Terminal 2: `tail -f app.log` y espere unos 10 segundos. ¿Qué pasa tras la rotación?
7. Repita con `tail -F app.log`. Anote la diferencia.

Ejercicios

1. Con el simulador en `--rafagas`, capture durante unos 60 segundos solo los `Login fallido` en `salidas/logins_vivo.txt`. Deténgalo con Ctrl+C y cuente las líneas.
2. Haga lo mismo sin `--line-buffered` y abra el archivo mientras corre. ¿Qué nota?
3. Use el formato web: `python3 simulador_log.py --formato web --archivo web.log --tasa 4` y siga solo las respuestas 5xx: `tail -f web.log | grep --line-buffered -E '" 5[0-9]{2} '`.
4. (Linux / WSL) `watch -n 2 "grep -c ERROR app.log"` refresca el conteo cada 2 s. En macOS, el equivalente simple es `while true; do clear; grep -c ERROR app.log; sleep 2; done`.

☐ Checkpoint: explique en una frase por qué en producción se prefiere `tail -F`. Screenshots de los ejercicios.

Actividad 4 — Filtrar por rango de tiempo (20 min)

Como la fecha va en formato `AAAA-MM-DD HH:MM:SS`, el orden alfabético coincide con el cronológico, así que `awk` puede comparar marcas de tiempo como texto.

Demostración guiada (ajuste las horas a las de su propio `app.log`; mírelas con `head -n 1` y `tail -n 1`)

```
# Eventos entre dos horas del mismo día
awk '$2 >= "11:00:00" && $2 < "11:10:00"' app.log

# Eventos de los últimos 5 minutos (Linux usa date -d, macOS usa date -v)
desde=$(date -d '5 minutes ago' '+%Y-%m-%d %H:%M:%S' 2>/dev/null || date -v-5
M '+%Y-%m-%d %H:%M:%S')
echo "Desde: $desde"
awk -v d="$desde" '($1" "$2) >= d' app.log > salidas/ultimos5min.txt

# Errores por minuto: se corta la hora a HH:MM
grep '\[ERROR\]' app.log | awk '{print substr($2,1,5)}' | uniq -c
```

Ejercicios

1. Guarde en `salidas/ventana.txt` los errores ocurridos en una ventana de 10 minutos que usted elija.
2. Con el conteo de errores por minuto, identifique el minuto con más errores. ¿Coincide con una ráfaga?
3. En `web.log` la fecha va como `15/Oct/2026:11:36:42`. ¿Por qué aquí ya no sirve comparar como texto entre días distintos? Proponga una solución.

☐ Checkpoint: puede responder “¿qué pasó entre las X y las Y?” con un solo comando. Screenshot de la demostración y los ejercicios.

Actividad 5 — Elaboración de scripts (35 min)

Todo lo anterior se convierte en dos scripts reutilizables: uno analiza el log por lotes y otro lo vigila durante un tiempo fijo.

Bloque 1 — `reporte_log.sh` (por lotes, 20 min). Complete los `TODO` de este esqueleto:

```
#!/usr/bin/env bash
# Uso: ./reporte_log.sh [archivo.log] [umbral_errores]
LOG="${1:-app.log}"          # parámetro 1, con valor por defecto
UMBRAL="${2:-20}"
mkdir -p reportes
REPORTE="reportes/reporte_$(date '+%Y%m%d_%H%M%S').txt"

# TODO 1: si "$LOG" no existe, mostrar un error en stderr (>&2) y salir con e
xit 1

ERRORES=$(grep -cE '\[(ERROR|CRITICAL)\]' "$LOG")

{
  echo "===== REPORTE DE $LOG ====="
  echo "Líneas totales: $(wc -l < "$LOG")"
  # TODO 2: conteo por nivel con un ciclo for
  # TODO 3: top 5 de servicios con más errores
  # TODO 4: ranking de IPs con login fallido
  # TODO 5: últimos 5 eventos CRITICAL
} > "$REPORTE"

echo "Reporte guardado en: $REPORTE"
# TODO 6: si ERRORES >= UMBRAL, imprimir ALERTA y salir con exit 2; si no, OK
```

Ejecución y prueba:

```
chmod +x reporte_log.sh
./reporte_log.sh app.log 50
echo "Código de salida: $?"
./reporte_log.sh no_existe.log; echo $?
```

En macOS y Git Bash, si `./script.sh` falla por permisos, ejecute `bash reporte_log.sh`.

Bloque 2 — `vigilar_log.sh` (por tiempo, 15 min). Vigila el log durante N segundos y guarda en `capturas/` las líneas que coinciden con un patrón. No usa `timeout` porque macOS no lo trae; en su lugar corre el `tail` en segundo plano y lo detiene tras un `sleep`:

```
#!/usr/bin/env bash
# Uso: ./vigilar_log.sh [archivo.log] [segundos] [patron]
LOG="${1:-app.log}"; SEGUNDOS="${2:-30}"; PATRON="${3:-ERROR|CRITICAL}"
mkdir -p capturas
SALIDA="capturas/captura_$(date '+%Y%m%d_%H%M%S').txt"

tail -n 0 -F "$LOG" 2>/dev/null | grep --line-buffered -E "$PATRON" | tee "$S
```

```
ALIDA" &
PID=$!
sleep "$SEGUNDOS"
kill "$PID" 2>/dev/null
pkill -P $$ tail 2>/dev/null
wait 2>/dev/null
echo "Líneas capturadas: $(wc -l < "$SALIDA")"
```

Prueba con dos terminales: en una `python3 simulador_log.py --tasa 5 --rafagas`; en la otra `./vigilar_log.sh app.log 30 "Login fallido"`.

Retos opcionales (para quien termine antes)

1. Agregue a `reporte_log.sh` una opción para analizar `web.log` y contar respuestas por código HTTP (`awk '{print $9}'`).
2. Haga que `vigilar_log.sh` emita un pitido (`printf '\a'`) cuando llegue un CRITICAL.
3. Programe el reporte cada 5 minutos con `cron` (Linux/WSL/macOS): `*/5 * * * * cd ~/lab_logs && ./reporte_log.sh app.log 50`.

☐ Checkpoint: `reportes/` contiene al menos un reporte, el script devuelve código 2 cuando se supera el umbral y `capturas/` tiene una captura con líneas. Adjunte en otro folder de onedrive sus scripts. Agregue algo que sea su “firma” o con lo que usted quiera distinguirse.